JSPM's
RAJARSHI SHAHU COLLEGE OF ENGINEERING
TATHAWADE, PUNE-33
(An Empowered Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune)

Mini Project

on

"Titanic Survival Prediction using Machine Learning"

-Submitted by-

Aniket B. Pathak **Roll No: CS3223**

Class and Division: TY Div B

Course: Machine Learning

Academic Year 2026-27

(Semester-I)

1. Title

Titanic Survival Prediction using Machine Learning

2. Abstract

The sinking of the RMS Titanic in 1912 caused the death of most of its passengers. Survival was not completely random: passengers of some groups, such as women and first-class travellers, had a much better chance of surviving. This mini project uses **supervised machine learning** to predict whether a Titanic passenger survived or not from details such as ticket class, gender, age, family members on board, fare and port of embarkation.

The dataset of 891 passengers was cleaned, explored with graphs and split into 80% training and 20% testing data. Four classification algorithms, Logistic Regression, Decision Tree, Random Forest and AdaBoost, were trained and compared using accuracy, precision, recall and F1-score. **Random Forest gave the best result with 82.7% accuracy** on unseen test data. Gender, fare and age were found to be the most important factors for survival.

3. Introduction

Machine Learning (ML) is a branch of Artificial Intelligence in which a computer learns patterns from data instead of being programmed with fixed rules. **Supervised learning** is the type of ML in which the model is trained on data that already contains the correct answer (called the label or target). After training, the model can predict the answer for new, unseen data.

When the answer belongs to a fixed set of categories, for example Survived or Died, the task is called **classification**. This project is a binary classification problem: the output is either 1 (survived) or 0 (died).

The Titanic dataset is one of the most popular beginner datasets in machine learning because it is small, easy to understand and contains a mix of numerical and categorical features, missing values and a clear target column.

4. Problem Statement

To build and compare supervised machine learning classification models that predict whether a passenger on the Titanic survived or not, using passenger details such as ticket class, gender, age, number of siblings/spouses and parents/children on board, fare paid and port of embarkation.

5. Objectives

- To understand and apply supervised learning (classification) on a real-world dataset.
- To clean the data by handling missing values and converting text columns into numbers.
- To explore the data using graphs and find which factors affected survival.
- To train four classification models: Logistic Regression, Decision Tree, Random Forest and AdaBoost.
- To compare the models using accuracy, precision, recall and F1-score and select the best one.
- To use the best model to predict survival for new passengers.

6. Data set and Libraries required

Data set

The Titanic passenger dataset (file: titanic_raw.csv) contains **891 passengers and 12 columns**. Source: https://github.com/datasciencedojo/datasets (titanic.csv).

Column	Description
Survived	Target: 0 = Died, 1 = Survived
Pclass	Ticket class: 1 (upper), 2 (middle), 3 (lower)
Name, Ticket, PassengerId	Identification details (not useful for prediction)
Sex	Gender of the passenger
Age	Age in years
SibSp	Number of siblings / spouses aboard
Parch	Number of parents / children aboard
Fare	Ticket fare paid
Cabin	Cabin number (mostly missing)
Embarked	Port of embarkation: S = Southampton, C = Cherbourg, Q = Queenstown

Libraries and tools

- **Python 3** with **Jupyter Notebook** as the working environment
- **pandas**: loading and cleaning the data
- **matplotlib** and **seaborn**: drawing graphs
- **scikit-learn**: train-test split, ML models and evaluation metrics

7. Approach/ Methodology

The project follows the standard supervised learning workflow:

Load data → Clean data → Explore (graphs) → Split into train/test → Train models → Evaluate and compare → Predict

- **Data cleaning:** missing Age values were filled with the median age, the 2 missing Embarked values with the most common port, and the Cabin column was dropped because about 77% of it was missing. PassengerId, Name and Ticket were removed as they do not help prediction.
- **Encoding:** machine learning models understand only numbers, so Sex (male = 0, female = 1) and Embarked (S = 0, C = 1, Q = 2) were converted to numeric values.
- **Exploratory data analysis:** bar charts and histograms were used to study survival by gender, class and age.
- **Train-test split:** 80% of the data (712 passengers) was used to train the models and 20% (179 passengers) was kept aside to test them on unseen data.
- **Model training:** Logistic Regression (uses a mathematical formula to separate the two classes), Decision Tree (asks a series of Yes/No questions to reach a decision), Random Forest (many decision trees vote and the majority answer wins) and AdaBoost (builds many small weak models one after another; each new model focuses on the passengers that the earlier models predicted wrongly, and all models are combined by a weighted vote).

- **Evaluation:** Accuracy (correct predictions out of all), Precision (when the model says Survived, how often it is right), Recall (out of all who really survived, how many the model found) and F1-score (balance of precision and recall).

8. Implementation

The complete code is given below, step by step. The same code, along with the dataset and result files, is available in the GitHub repository:

GitHub: https://github.com/aniket123pathak/ML_mini_project

1. Importing the libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix, ConfusionMatrixDisplay
```

2. Loading the dataset

```
df = pd.read_csv("titanic_raw.csv")
print("Rows, Columns:", df.shape)
df.head()
```

3. Data cleaning

First the number of missing values in each column is checked:

```
df.isnull().sum()
```

Column	Missing values	Action taken
Age	177	Filled with median age
Embarked	2	Filled with most common port
Cabin	687	Column dropped

Missing values are then filled, and the Cabin column is dropped:

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df = df.drop(columns=["Cabin"])


print("Missing values left:", df.isnull().sum().sum())
```

Text columns are converted to numbers (Sex: male = 0, female = 1; Embarked: S = 0, C = 1, Q = 2) and the cleaned data is saved as titanic_analysis.csv:

```
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
df["Embarked"] = df["Embarked"].map({"S": 0, "C": 1, "Q": 2})


# Save the cleaned data
clean = df.drop(columns=["PassengerId", "Name", "Ticket"])
clean.to_csv("titanic_analysis.csv", index=False)
clean.head()
```

4. Exploratory data analysis

Only about 38% of the passengers survived and 62% died. Women had a survival rate of about 74% compared with about 19% for men. Survival also fell with ticket class: about 63% in first class, 47% in second class and 24% in third class.

```
plt.figure(figsize=(4, 3.5))
sns.countplot(data=df, x="Survived", palette=["#D9534F", "#4C9F70"])
plt.xticks([0, 1], ["Died", "Survived"])
plt.title("How many survived?")
plt.show()
print(df["Survived"].value_counts(normalize=True).round(2) * 100)
```

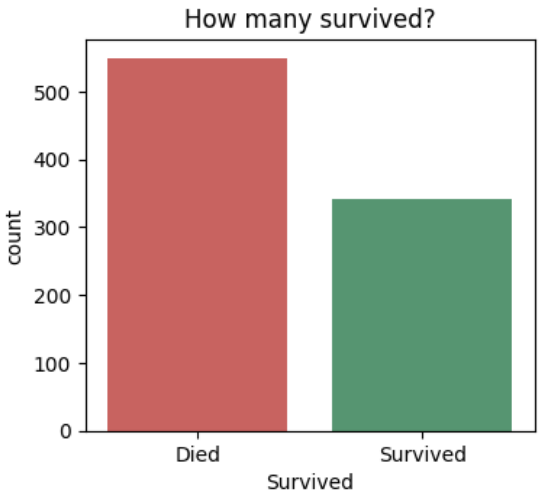


Figure 1: Survival count (0 = Died, 1 = Survived)

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
sns.barplot(data=df, x="Sex", y="Survived", ax=ax[0], palette="Set2")
ax[0].set_xticklabels(["Male", "Female"]); ax[0].set_title("Survival rate by
Gender")
sns.barplot(data=df, x="Pclass", y="Survived", ax=ax[1], palette="Set2")
ax[1].set_title("Survival rate by Ticket Class")
plt.tight_layout(); plt.show()
```

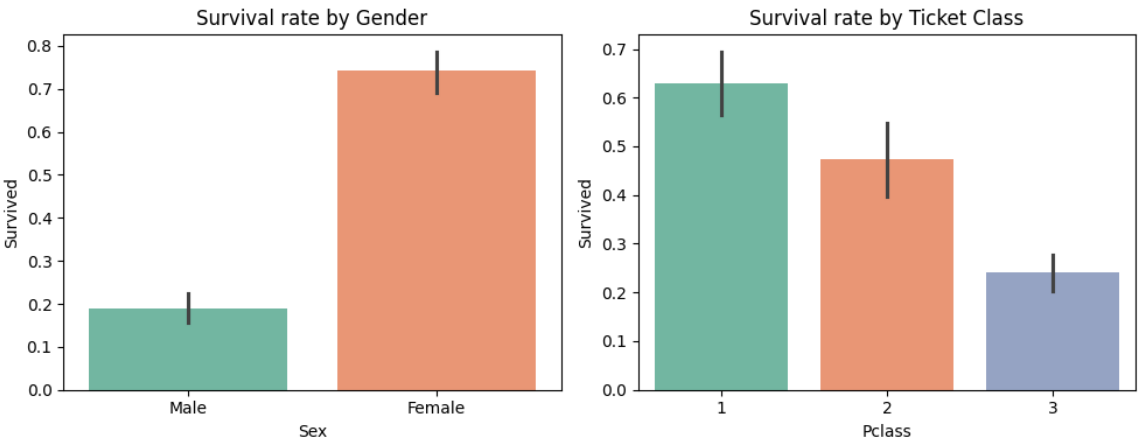


Figure 2: Survival rate by gender and by ticket class

```
plt.figure(figsize=(7, 4))
sns.histplot(data=df, x="Age", hue="Survived", bins=25, palette=["#D9534F",
"#4C9F70"])
plt.title("Age vs Survival")
plt.show()
```

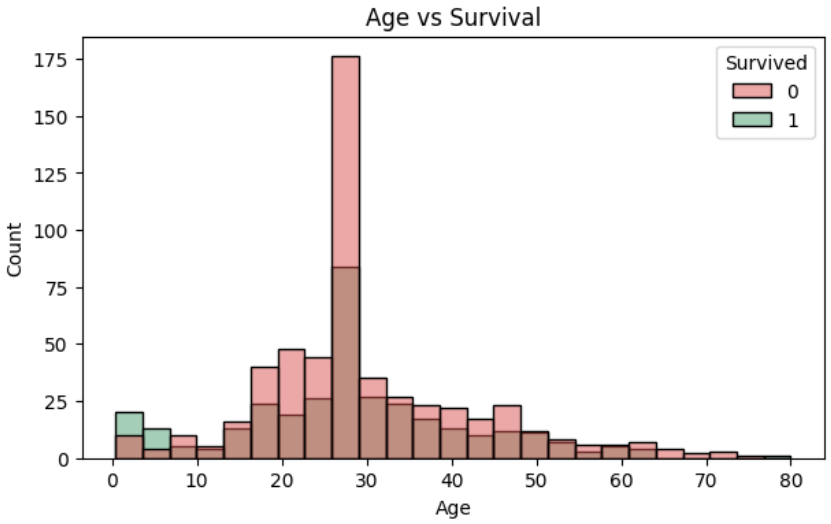


Figure 3: Age distribution of passengers who survived and died

5. Train-test split

80% of the data is used for training and 20% for testing.

```
X = df[["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]]
y = df["Survived"]


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print("Training rows:", len(X_train), "| Testing rows:", len(X_test))
```

6. Training and evaluating the models

Four supervised classification models are trained on the training data and tested on the unseen test data. The results are saved in model_comparison.csv.

```
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(max_depth=4, random_state=42),
```

```
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=50, random_state=42),
}

rows = []
predictions = {}
for name, model in models.items():
    model.fit(X_train, y_train)          # learn from training data
    pred = model.predict(X_test)         # predict on unseen test data
    predictions[name] = pred
    rows.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1-Score": f1_score(y_test, pred),
    })

results = pd.DataFrame(rows).round(4).sort_values(["Accuracy", "F1-Score"],
ascending=False).reset_index(drop=True)
results.to_csv("model_comparison.csv", index=False)
results
```

7. Model comparison chart

```
plt.figure(figsize=(7, 3.5))
sns.barplot(data=results, x="Accuracy", y="Model", palette="Blues_r")
plt.xlim(0.5, 1); plt.title("Model Accuracy Comparison")
plt.show()
```

8. Best model and confusion matrix

```
best_name = results.loc[0, "Model"]
best_model = models[best_name]
print("Best model:", best_name, "| Accuracy:", results.loc[0, "Accuracy"])

ConfusionMatrixDisplay.from_predictions(y_test, predictions[best_name],
                                        display_labels=["Died", "Survived"],
cmap="Blues")
plt.title("Confusion Matrix - " + best_name)
plt.show()
```

9. Decision tree diagram

```
small_tree = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train,
y_train)
plt.figure(figsize=(16, 7))
plot_tree(small_tree, feature_names=X.columns, class_names=["Died", "Survived"],
filled=True, fontsize=9)
plt.show()
```

10. Feature importance (Random Forest)

```
imp = pd.Series(models["Random Forest"].feature_importances_,
index=X.columns).sort_values()
imp.plot(kind="barh", color="#5B8DB8", figsize=(6, 3.5), title="Feature Importance")
plt.show()
```

11. Saving the predictions

The predictions of the best model on the test passengers are saved in survival_predictions.csv.

```
out = df.loc[X_test.index, ["PassengerId", "Name"]].copy()
out["Actual"] = y_test.map({0: "Died", 1: "Survived"})
out["Predicted"] = pd.Series(predictions[best_name], index=X_test.index).map({0:
"Died", 1: "Survived"})
out["Correct"] = (out["Actual"] == out["Predicted"]).map({True: "Yes", False: "No"})
out = out.sort_values("PassengerId").reset_index(drop=True)
out.to_csv("survival_predictions.csv", index=False)
print("Correct predictions:", (out["Correct"] == "Yes").sum(), "out of", len(out))
out.head(10)
```

12. Predicting for a new passenger

```
def predict_passenger(pclass, sex, age, sibsp, parch, fare, embarked):
    # sex: "male"/"female" | embarked: "S"/"C"/"Q"
    row = pd.DataFrame([[pclass, 0 if sex == "male" else 1, age, sibsp, parch, fare,
                         {"S": 0, "C": 1, "Q": 2}[embarked]]], columns=X.columns)
    return "Survived" if best_model.predict(row)[0] == 1 else "Died"


print("1st class woman, age 28 :", predict_passenger(1, "female", 28, 0, 0, 80,
"S"))
print("3rd class man, age 30   :", predict_passenger(3, "male", 30, 0, 0, 8, "S"))
print("2nd class girl, age 8   :", predict_passenger(2, "female", 8, 1, 1, 26, "S"))
```

9. Results

All four models were tested on the same 179 unseen passengers.

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.8268	0.8116	0.7568	0.7832
Logistic Regression	0.7989	0.7714	0.7297	0.7500
AdaBoost	0.7989	0.7879	0.7027	0.7429
Decision Tree	0.7989	0.8393	0.6351	0.7231

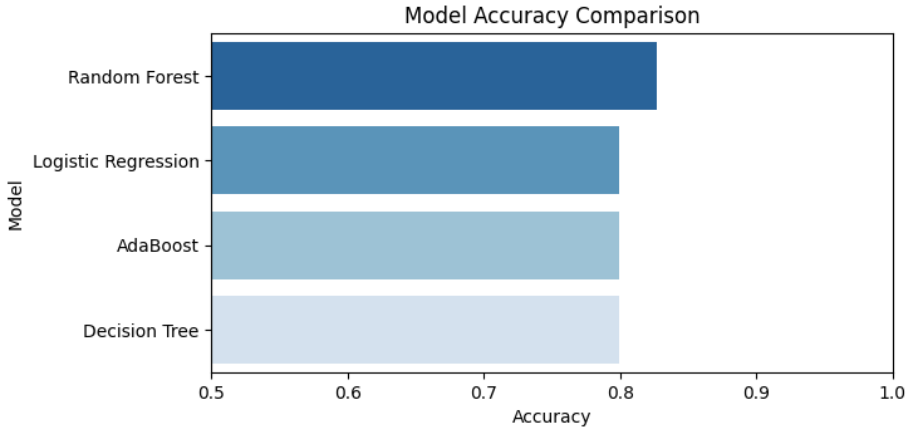


Figure 4: Accuracy comparison of the four models

Random Forest is the best model with **82.7% accuracy**: 148 of the 179 test passengers were predicted correctly. Logistic Regression, AdaBoost and Decision Tree all reached 79.9%; in the table they are ordered by F1-score (0.75, 0.74 and 0.72).

Confusion matrix (Random Forest)

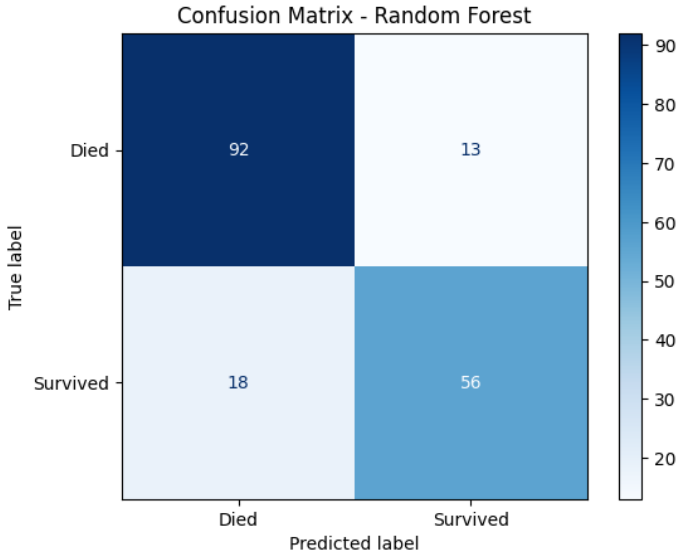


Figure 5: Confusion matrix of the Random Forest model

Of the 179 test passengers, 92 who died and 56 who survived were predicted correctly (148 correct). The model wrongly predicted 13 passengers as survived when they died, and 18 as died when they actually survived.

Decision tree

The decision tree is easy to understand because it shows the questions the model asks, such as whether the passenger is female or male, the ticket class and the age. It is shown below with depth 3.

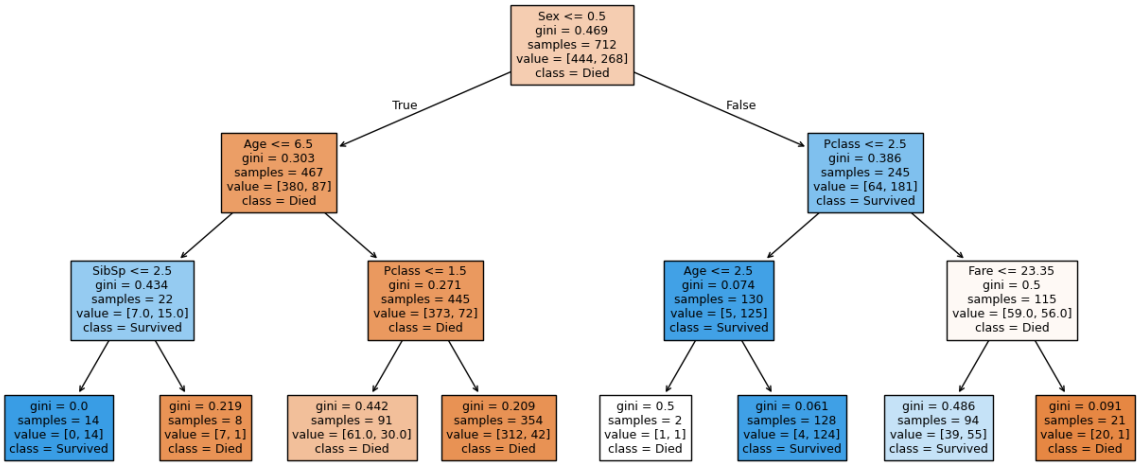


Figure 6: Decision tree (depth 3)

Important features

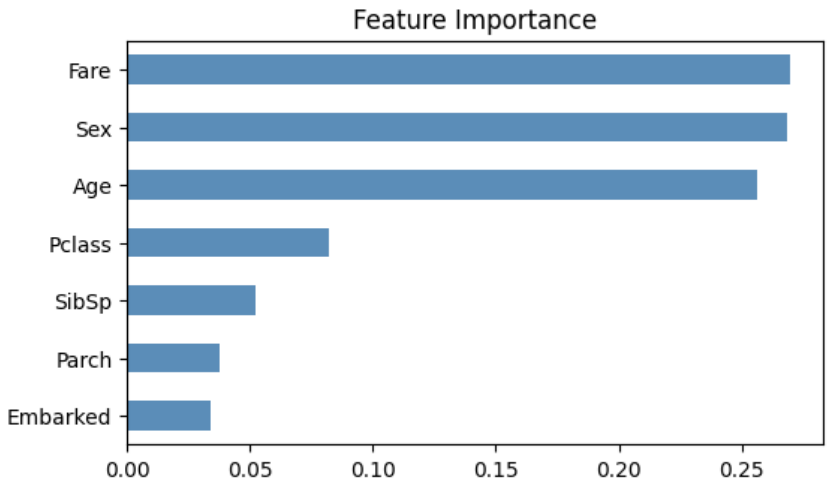


Figure 7: Feature importance from the Random Forest model

Fare (0.27), Sex (0.27) and Age (0.26) are the three most important features, followed by Pclass (0.08). In the graphs, gender showed the strongest effect on survival, which matches the "women and children first" rule followed during the evacuation.

Prediction for new passengers

Passenger	Predicted result
1st class woman, age 28	Survived
3rd class man, age 30	Died
2nd class girl, age 8	Survived

10. Conclusion

In this project, supervised machine learning (classification) was used to predict Titanic passenger survival. After cleaning the data and exploring it, four models were trained and tested on unseen passengers. **Random Forest performed best with 82.7% accuracy**, ahead of Logistic Regression, AdaBoost and Decision Tree (79.9% each).

The analysis showed that gender, fare and age are the most important factors, and that women, children and first-class passengers had a much better chance of survival.

Limitations: the dataset is small (891 passengers) and the models were used with basic settings. With only 179 test passengers, a few passengers change the accuracy by a visible amount, so the gap between Random Forest and the other models should not be over-interpreted. **Future scope:** trying more algorithms such as KNN and SVM, tuning model parameters and creating new features such as family size and passenger title.

GitHub repository: https://github.com/aniket123pathak/ML_mini_project